

MongoDB — Operations Guide

Self-hosted MongoDB serving TDP Saga and any future applications.

Host	32.192.131.130:27017
Version	MongoDB 8.0.28 (Ubuntu 24.04)
Data directory	/var/lib/mongodb (persistent EBS volume)
Authentication	Required — every connection needs credentials
Server access	ssh -i ~/.ssh/punjab_saga.pem ubuntu@32.192.131.130

Credentials are not in this file. Wherever you see `<ADMIN_PASSWORD>` or `<APP_PASSWORD>`, substitute the real value from your password manager. Never commit real credentials to this repository.

1. Logging in as admin

The `admin` account has full control over every database. Use it for setup and maintenance only — never in an application.

From the server:

```
mongosh -u admin -p '<ADMIN_PASSWORD>' --authenticationDatabase admin
```

From your laptop, or any other machine:

```
mongosh "mongodb://admin:<ADMIN_PASSWORD>@32.192.131.130:27017/?authSource=admin"
```

The same string works in **MongoDB Compass**.

Once connected:

```
show dbs // list all databases
use apsaga // switch to a database
show collections // list its collections
db.grievances.countDocuments() // count documents
exit
```

2. Adding a new application

Three steps. The example below uses an app called **telangana** — substitute your own name throughout.

Step 1 — Generate a password

```
openssl rand -hex 24
```

Save the output. Hex only, so it never needs URL-encoding inside a connection string.

Step 2 — Create the application's user

```
mongosh -u admin -p '<ADMIN_PASSWORD>' --authenticationDatabase admin
```

```
use telangana
db.createUser({
  user: "telangana_app",
  pwd: "<GENERATED_PASSWORD>",
  roles: [ { role: "readWrite", db: "telangana" } ]
})
exit
```

Step 3 — Configure the application

In that application's **.env**:

```
MONGODB_URI="mongodb://telangana_app:
<GENERATED_PASSWORD>@32.192.131.130:27017/telangana?authSource=telangana"
DB_NAME="telangana"
```

Start the app. **The database is created automatically on its first write** — use **telangana** alone does not create it, so it won't appear in **show dbs** or Compass until real data is written. That's expected, not a failure.

3. Connection string reference

```
mongodb://telangana_app:PASSWORD@32.192.131.130:27017/telangana?
authSource=telangana
└── user ─┘ └─ pass ─┘ └── server ───┘ └─ database ┘ └── see below
└──┘
```

authSource tells MongoDB **where to verify the password**, which is not always where the data lives. This is the most common source of **Authentication failed** errors.

Connecting as	authSource
admin	admin
Any application user that application's own database name	

Current applications

Application	Database	User	authSource
TDP Saga	apsaga	apsaga_app	apsaga

4. Common tasks

All run from an admin shell.

Task	Command
List databases	<code>show dbs</code>
List users on a database	<code>use <db> then db.getUsers()</code>
Create an app user	<code>db.createUser({user, pwd, roles: [{role:"readWrite", db:"<db>"}]})</code>
Change a password	<code>use <db> then db.changeUserPassword("<user>", "<new>")</code>
Remove a user	<code>use <db> then db.dropUser("<user>")</code>
Database size	<code>use <db> then db.stats()</code>
Service status	<code>sudo systemctl status mongod</code>
Restart the service	<code>sudo systemctl restart mongod</code>
Server logs	<code>sudo tail -50 /var/log/mongodb/mongod.log</code>

5. Backup and restore

Manual backup

```
mongodump -u admin -p '<ADMIN_PASSWORD>' --authenticationDatabase admin \
--db=apsaga --gzip --archive=/tmp/apsaga-$(date +%Y%m%d).archive.gz
```

Restore

```
mongorestore -u admin -p '<ADMIN_PASSWORD>' --authenticationDatabase admin \
--gzip --archive=/tmp/apsaga-20260807.archive.gz
```

Add `--drop` to wipe each collection before restoring — use it when replacing existing data, and only deliberately.

Scheduled backups

Script at `~/mongo-backup.sh` on the server. It queries the server for the list of databases rather than using a fixed list, so **applications added later are included automatically** with no edit required.

```
/var/backups/mongo/<database>/<database>-YYYYMMDD-HHMMSS.archive.gz
```

Check it is running:

```
crontab -l  
tail ~/mongo-backup.log
```

Backups currently live on the same disk as the database. They protect against deleted or corrupted data, but not against loss of the server itself. Copying them to S3 is an outstanding improvement.

6. Security

Rules

- **One user per database.** Each application's credential opens only its own database, so a leaked `.env` cannot reach another application's data.
- **Never put the `admin` password in an application.** It can drop every database on the server.
- **Never commit a `.env` or a real password** to this repository.
- **`readWrite` is enough for applications.** They do not need to create users or drop databases.

Verifying authentication is enforced

From any machine other than the server:

```
# must fail  
mongosh "mongodb://32.192.131.130:27017/apsaga" --eval  
'db.grievances.countDocuments()'
```

```
# must succeed
```

```
mongosh "mongodb://apsaga_app:<APP_PASSWORD>@32.192.131.130:27017/apsaga?authSource=apsaga" \
--eval 'db.grievances.countDocuments()'
```

An unauthenticated client can open a socket — MongoDB completes the protocol handshake before checking credentials — but every operation is refused. Seeing "connected" without credentials is normal; being able to *read* would not be.